

AUTH SYSTEM

Developer FAQ Document

WHY NEXT.JS / REACT?

Why We Use Next.js Instead of Bootstrap + Plain HTML

The Question

Why are we using Node.js and React (Next.js) instead of simple Bootstrap and plain HTML for the frontend of this application?

Short Answer

Bootstrap and HTML are tools for displaying information. Next.js is a tool for building applications. This project is an application with user accounts, protected pages, and security requirements. That is exactly the problem Next.js was designed to solve. Bootstrap cannot protect a page before it loads. Next.js can.

1. Think About What This Application Actually Does

This is not a simple website like a restaurant menu or a contact page. This is an application where:

- Users create accounts and log in with four different methods
- Pages must be completely hidden from people who are not logged in
- Content changes depending on who is logged in
- Sessions must be managed securely using HttpOnly cookies
- Tokens must refresh automatically in the background without the user noticing
- Every security event must be recorded in an audit log

The moment your application has user accounts and protected pages, plain HTML and Bootstrap hit a hard wall. The following sections explain exactly where and why.

2. Problem 1 — HTML Files Cannot Protect Themselves

Imagine your dashboard page is a file called `dashboard.html` sitting on your server. Here is what happens when a user who is NOT logged in tries to visit it:

With Plain HTML + Bootstrap	With Next.js
Step 1: Browser downloads <code>dashboard.html</code> instantly — no questions asked	Step 1: Request hits the Next.js server first
Step 2: Page content flashes on the screen	Step 2: Server checks the login cookie BEFORE rendering anything
Step 3: JavaScript runs and checks if the user is logged in	Step 3: If not logged in, server sends a redirect instantly
Step 4: JavaScript redirects to the login page	Step 4: The page HTML never reaches the browser at all
PROBLEM: The page content was briefly visible. The file also exists as a downloadable URL that anyone can access directly.	RESULT: No flash. No exposure. The protection happens on the server before any content is sent.

This Is the Single Biggest Reason

You simply cannot protect a page before it loads using plain HTML. The file exists on the server and anyone can request it. Next.js runs a server that decides what each person is allowed to receive before sending anything.

3. Problem 2 — Every HTML Page Starts Blank and Knows Nothing

In plain HTML, every page loads completely empty. It has no idea who you are. It has to figure everything out from scratch on every single page load.

With Plain HTML + Bootstrap	With Next.js
Step 1: Page loads completely blank	Step 1: Browser requests the page
Step 2: Bootstrap CSS loads and styles the empty page	Step 2: Next.js server fetches user data from FastAPI simultaneously
Step 3: JavaScript runs and calls the API to find out who is logged in	Step 3: Server builds the complete HTML page with user data already inside
Step 4: API responds after 200 to 500 milliseconds	Step 4: Browser receives one complete ready page
Step 5: JavaScript updates the page with the user data	Step 5: User sees the full page immediately with no waiting
Result: Every page flickers. Every page makes an extra API call. Users see loading spinners constantly. Feels slow.	Result: No flicker. No spinners for content. Feels instant. One round trip to the server instead of two.

4. Problem 3 — Copy and Paste Hell

Every page in a plain HTML project needs the same code repeated in every single file. This becomes a maintenance nightmare as the project grows.

With Plain HTML + Bootstrap	With Next.js
You have 8 pages. Every single page file needs: navigation bar HTML, Bootstrap CSS link, custom CSS link, auth check JavaScript, token refresh JavaScript, error handling JavaScript, logout function.	Navigation bar lives in ONE file: Navbar.tsx. Auth check lives in ONE file: middleware.ts. Token refresh lives in ONE file: useTokenRefresh.ts. Error handling lives in ONE file: ErrorBanner.tsx.
Designer wants to change the navigation bar. You edit 8 files.	Designer wants to change the navigation bar. You edit 1 file. All 8 pages update automatically.
You find a bug in the auth check code. You fix it in 8 files. If you miss one file, that page has the bug forever.	You find a bug in the auth check code. You fix it in 1 file. Every page is fixed.
You add a 9th page. You copy all the repeated code again into the new file.	You add a 9th page. You create 1 new file. Everything else is included automatically.

5. Problem 4 — JavaScript in HTML Files Is Visible to Everyone

When you write JavaScript inside an HTML file or link a .js file, that code is sent to the browser. Anyone can press F12 and read every line of your authentication logic.

With Plain HTML + Bootstrap	With Next.js
All your JavaScript is sent to the browser. Anyone presses F12, opens the Sources tab, and reads every line of your auth logic.	Server components run ONLY on the server. Their code is never sent to the browser. The browser only receives the final HTML result.
Attacker can see: how you check login status, what API calls you make, how routing works, what error messages exist, what validation rules you use.	The auth logic, cookie checking, and user data fetching never appear in the browser developer tools. Only the rendered HTML is visible.

6. Problem 5 — SEO — Google Cannot Read Dynamic Content

Search Engine Optimisation means Google and other search engines can find and rank your pages. This is critical for any application that wants users to discover it through search.

With Plain HTML + Bootstrap	With Next.js
Static pages are fine for SEO. But the moment you add JavaScript to fetch and display data dynamically, Google sees only an empty container div — not the actual content.	Next.js renders the complete HTML on the server before sending it to anyone including Google. Google receives a fully populated page with all content visible.
Google sees: <code><div id='content'></div></code> . An empty box. Your content is invisible to search engines.	Google sees: <code><h1>Welcome to AuthSystem</h1><p>The most secure login system</p></code> . Every word is readable and indexable.
Result: Your application does not appear in Google search results. Users cannot find you.	Result: Your pages appear in Google search results. Users can find you organically.

7. Visual Summary — The Core Difference

Feature	Plain HTML + Bootstrap	Next.js + React
What it is	A collection of files sitting on a hard drive	A running server that thinks before responding
Page protection	After the page loads in browser	Before the page is sent to browser
Who can access files	Anyone who knows the URL	Only people the server allows
Repeated code	In every HTML file	Written once, used everywhere
Loading experience	Flicker and spinners on every page	Content appears immediately
Auth logic visibility	Visible to anyone via F12	Stays private on the server
Google SEO	Good for static content only	Full content readable by Google
Best for	Marketing sites, landing pages, blogs	Applications with user accounts
Complexity	Simple to start	More to learn upfront
Cost	Free static hosting	Needs a server or Vercel

8. What About Bootstrap — Is It Gone?

Bootstrap is not replaced by Next.js. They solve completely different problems. They are not even in the same category.

Tool	What Problem It Solves	Our Equivalent
Bootstrap	Makes pages look good with pre-built CSS styles	Tailwind CSS — same job, more efficient
Next.js	Builds the application structure, routing, server rendering	Next.js — no equivalent needed, it IS the framework
Plain HTML	Defines the page structure	JSX in React components — compiles to plain HTML
Vanilla JavaScript	Makes pages interactive	React — same job but organised into reusable components

We replaced Bootstrap with Tailwind CSS — not with Next.js. Tailwind does exactly what Bootstrap does: it gives you pre-built CSS classes to make things look good. The difference is that Tailwind only includes the CSS you actually use, making the final file much smaller.

You could use Bootstrap instead of Tailwind in this project and everything would still work perfectly. The Next.js framework would stay because Bootstrap cannot replace what Next.js does at the server level. They are completely separate concerns.

9. When to Use Each — The Honest Guide

There is no universally correct answer. The right tool depends on what you are building.

Use Plain HTML + Bootstrap When	Use Next.js + React When
Building a simple informational website	Building an application with user accounts
No user accounts or login needed	Pages must be protected from unauthenticated access
Small project with 1 to 3 pages	Project will grow to many pages over time
You need something working in one day	Multiple developers will work on the codebase
Client just needs a brochure website	Performance and page load speed matter
Content never changes based on the user	Content is personalised per logged-in user
Budget is very limited for hosting	SEO matters for dynamic data pages
No SEO requirements	You want Google to index your application pages

10. The Final Answer in Plain English

Bootstrap and HTML are tools for displaying information. They are perfect for a company homepage, a product catalogue, or a blog.

Next.js and React are tools for building applications — software that has users, protects data, manages sessions, and responds differently based on who is asking.

You would not use a screwdriver to hammer a nail. Both are valid tools. They just solve completely different problems.

This project is an authentication system. It has user accounts, protected pages, secure cookies, session management, and audit logging. That combination of requirements is exactly the problem Next.js was designed to solve. Plain HTML and Bootstrap were not designed for this and cannot do it securely.

Remember This

Node.js is the engine. Next.js is the framework that runs on that engine. React is the library for building UI components. Bootstrap is a CSS styling tool. Tailwind CSS is also a CSS styling tool. These are all different layers solving different problems. None of them replace each other — they each do a specific job.